

Cours PHP Complet

Syntaxe • Chaînes • POO • Sécurité • Débogage • Superglobales •
Téléversement • Connexion BDD

Environnement : LAMP (Linux, Apache, MySQL, PHP)

Outils : phpMyAdmin, VS Code, SQLTools

Table des matières

1	Syntaxe de base et structures du langage	3
1.1	Structure d'un fichier PHP	3
1.2	Variables et types	3
1.3	Conditions	3
1.4	Boucles	4
1.5	Fonctions	4
2	Chaînes de caractères	6
2.1	Quotes simples vs doubles	6
2.2	Concaténation	6
2.3	Fonctions mb* pour l'UTF-8	6
2.4	Fonctions utiles sur les chaînes	7
3	Programmation Orientée Objet (POO)	8
3.1	Créer une classe et un objet	8
3.2	Encapsulation	9
3.3	Héritage	9
4	Gestion des entrées utilisateur et sécurité	11
4.1	Récupérer les données d'un formulaire	11
4.2	Échappement et sécurité	11
4.3	Validation des données	12
5	Débogage avec var_dump	13
5.1	Les outils de débogage	13
5.2	Fonctions de débogage personnalisées	13
5.3	Activer l'affichage des erreurs	14
6	Superglobales	15
7	Téléversement de fichiers	16
7.1	Formulaire HTML	16
7.2	Traitement sécurisé du fichier	16
7.3	Téléverser plusieurs fichiers	17
8	Connexion à la base de données avec LAMP	18
8.1	Prérequis : vérifier que LAMP est actif	18
8.2	Approche 1 — Fichier de connexion PHP	18
8.2.1	Créer le fichier connexion.php	18
8.2.2	Tester la connexion dans le navigateur	19
8.2.3	Utiliser la connexion dans les autres fichiers	19
8.2.4	Afficher une table dans le navigateur	20
8.3	Approche 2 — Extension SQLTools dans VS Code	21
8.3.1	Extensions à installer	21
8.3.2	Configurer la connexion	21
8.3.3	Explorer les tables	21
8.3.4	Exécuter des requêtes SQL depuis VS Code	22
8.3.5	Résumé comparatif	22

Récapitulatif général	23
-----------------------	----

1 Syntaxe de base et structures du langage

1.1 Structure d'un fichier PHP

```
1 <?php
2 // Tout code PHP commence par <?php
3 // Commentaire sur une ligne
4
5 /* Commentaire
6    multi-lignes */
7
8 echo "Bonjour le monde !";
9 // La balise fermante ?> est optionnelle dans un fichier 100%
   PHP
```

Listing 1 – Structure de base d'un fichier PHP

Attention

La balise fermante ?> est optionnelle dans un fichier 100% PHP. Il vaut mieux l'omettre pour éviter les espaces parasites en fin de fichier.

1.2 Variables et types

```
1 <?php
2 // Les variables commencent toujours par $
3 $nom      = "Jean";           // String
4 $age       = 25;               // Integer
5 $taille    = 1.75;            // Float
6 $actif     = true;            // Boolean
7 $rien      = null;            // Null
8
9 // Afficher le type et la valeur
10 var_dump($age);               // int(25)
11 var_dump($actif);             // bool(true)
```

Listing 2 – Variables et types en PHP

1.3 Conditions

```
1 <?php
2 $age = 20;
3
4 // if / elseif / else
5 if ($age >= 18) {
6     echo "Majeur";
7 } elseif ($age >= 16) {
8     echo "Presque majeur";
9 } else {
10    echo "Mineur";
```

```
11 }
12
13 // Operateur ternaire
14 $statut = ($age >= 18) ? "Majeur" : "Mineur";
15
16 // Switch
17 $jour = "Lundi";
18 switch ($jour) {
19     case "Lundi":
20         echo "Debut de semaine";
21         break;
22     case "Vendredi":
23         echo "Fin de semaine";
24         break;
25     default:
26         echo "Milieu de semaine";
27 }
```

Listing 3 – Structures conditionnelles

1.4 Boucles

```
1 <?php
2 // Boucle for
3 for ($i = 0; $i < 5; $i++) {
4     echo $i . "<br>";
5 }
6
7 // Boucle while
8 $compteur = 0;
9 while ($compteur < 5) {
10     echo $compteur . "<br>";
11     $compteur++;
12 }
13
14 // Boucle foreach (pour les tableaux)
15 $fruits = ["Pomme", "Banane", "Cerise"];
16 foreach ($fruits as $fruit) {
17     echo $fruit . "<br>";
18 }
19
20 // foreach avec cle => valeur
21 $employe = ["nom" => "Dupont", "prenom" => "Jean"];
22 foreach ($employe as $cle => $valeur) {
23     echo "$cle : $valeur <br>";
24 }
```

Listing 4 – Les boucles en PHP

1.5 Fonctions

```
1 <?php
2 // Declaration
3 function direBonjour($prenom) {
4     return "Bonjour " . $prenom . " !";
5 }
6
7 echo direBonjour("Marie"); // Bonjour Marie !
8
9 // Valeur par défaut
10 function calculerTaxe($prix, $taux = 0.20) {
11     return $prix * $taux;
12 }
13
14 echo calculerTaxe(100); // 20
15 echo calculerTaxe(100, 0.10); // 10
```

Listing 5 – Déclaration et appel de fonctions

2 Chaînes de caractères

2.1 Quotes simples vs doubles

```
1 <?php
2 $nom = "Jean";
3
4 // Double quotes : les variables SONT interpretees
5 echo "Bonjour $nom !";           // Bonjour Jean !
6 echo "Bonjour {$nom} !";        // Bonjour Jean ! (recommande)
7
8 // Simple quotes : les variables NE SONT PAS interpretees
9 echo 'Bonjour $nom !';          // Bonjour $nom ! (litteral)
10
11 // Caracteres speciaux (double quotes uniquement)
12 echo "Ligne 1\nLigne 2";        // \n = saut de ligne
13 echo "Colonne1\tColonne2";     // \t = tabulation
```

Listing 6 – Différence entre quotes simples et doubles

2.2 Concaténation

```
1 <?php
2 $prenom = "Jean";
3 $nom     = "Dupont";
4
5 // L'operateur de concatenation est le point .
6 $nomComplet = $prenom . " " . $nom;
7 echo $nomComplet; // Jean Dupont
8
9 // Concatenation avec affectation
10 $message = "Bonjour ";
11 $message .= $nomComplet;
12 echo $message; // Bonjour Jean Dupont
```

Listing 7 – Concaténation de chaînes

2.3 Fonctions mb* pour l'UTF-8

Attention

Les fonctions classiques comme `strlen()` comptent les **octets**, pas les caractères. Avec les accents (UTF-8), il faut obligatoirement utiliser les fonctions `mb_*` (multi-byte).

```
1 <?php
2 $texte = "Hello world";
3
4 // Longueur
```

```
5 echo strlen($texte);           // Peut etre incorrect en UTF
   -8
6 echo mb_strlen($texte, 'UTF-8'); // Correct
7
8 // Majuscule / minuscule
9 echo mb_strtoupper($texte, 'UTF-8');
10 echo mb_strtolower($texte, 'UTF-8');
11
12 // Extraire une sous-chaine
13 echo mb_substr($texte, 0, 5, 'UTF-8');
14
15 // Trouver la position
16 echo mb_strpos($texte, "world", 0, 'UTF-8');
```

Listing 8 – Fonctions mb* pour l'UTF-8

2.4 Fonctions utiles sur les chaînes

```
1 <?php
2 $texte = "  Bonjour le monde !  ";
3
4 echo trim($texte);           // Supprime les
   espaces
5 echo strtoupper($texte);     // MAJUSCULES
6 echo strtolower($texte);     // minuscules
7 echo str_replace("monde", "PHP", $texte); // Remplacement
8 echo strlen($texte);        // Longueur
9 echo strpos($texte, "monde"); // Position
10
11 // Diviser en tableau
12 $csv = "Jean,Marie,Paul";
13 $noms = explode(",", $csv); // ["Jean","Marie","Paul"]
14
15 // Assembler un tableau en chaine
16 echo implode(" - ", $noms); // Jean - Marie - Paul
```

Listing 9 – Fonctions courantes sur les chaînes

3 Programmation Orientée Objet (POO)

3.1 Créer une classe et un objet

```
1 <?php
2 class Employe {
3     public string $nom;
4     public string $prenom;
5     private int $salaire;    // Prive = accessible uniquement
6                               dans la classe
7
8     // Constructeur
9     public function __construct(string $nom, string $prenom, int
10        $salaire) {
11         $this->nom      = $nom;
12         $this->prenom   = $prenom;
13         $this->salaire  = $salaire;
14     }
15
16     public function sePresenter(): string {
17         return "Je suis {$this->prenom} {$this->nom}.";
18     }
19
20     // Getter
21     public function getSalaire(): int {
22         return $this->salaire;
23     }
24
25     // Setter avec validation
26     public function setSalaire(int $salaire): void {
27         if ($salaire > 0) {
28             $this->salaire = $salaire;
29         }
30     }
31 }
32
33 // Instanciation
34 $employe1 = new Employe("Dupont", "Jean", 2000);
35 echo $employe1->sePresenter();    // Je suis Jean Dupont.
36 echo $employe1->getSalaire();    // 2000
37 echo $employe1->setSalaire(2500);
38 echo $employe1->getSalaire();    // 2500
```

Listing 10 – Classe, constructeur et méthodes

3.2 Encapsulation

Info

L'encapsulation consiste à **protéger les données** d'une classe en contrôlant leur accès depuis l'extérieur. On utilise les modificateurs `public`, `private` et `protected`.

```
1 <?php
2 class CompteBancaire {
3     private float $solde;
4
5     public function __construct(float $soldeInitial) {
6         $this->solde = $soldeInitial;
7     }
8
9     public function getSolde(): float {
10         return $this->solde;
11     }
12
13     public function deposer(float $montant): void {
14         if ($montant > 0) {
15             $this->solde += $montant;
16         }
17     }
18
19     public function retirer(float $montant): void {
20         if ($montant > 0 && $montant <= $this->solde) {
21             $this->solde -= $montant;
22         } else {
23             echo "Solde insuffisant !";
24         }
25     }
26 }
27
28 $compte = new CompteBancaire(1000);
29 $compte->deposer(500);
30 echo $compte->getSolde();    // 1500
31 $compte->retirer(200);
32 echo $compte->getSolde();    // 1300
33
34 // $compte->solde = 9999;    INTERDIT (propriete privatee)
```

Listing 11 – Encapsulation avec getters et setters

3.3 Héritage

```
1 <?php
2 // Classe parente
3 class Personne {
4     public string $nom;
5     public string $prenom;
```

```
6
7     public function __construct(string $nom, string $prenom) {
8         $this->nom      = $nom;
9         $this->prenom = $prenom;
10    }
11
12    public function sePresenter(): string {
13        return "Je suis {$this->prenom} {$this->nom}.";
14    }
15 }
16
17 // Classe enfant
18 class Manager extends Personne {
19     private string $departement;
20
21     public function __construct(string $nom, string $prenom,
22                                string $dept) {
23         parent::__construct($nom, $prenom); // Appel
24         constructeur parent
25         $this->departement = $dept;
26     }
27
28     // Surcharge (override)
29     public function sePresenter(): string {
30         return parent::sePresenter()
31             . " Je manage le departement {$this->departement}."
32             ;
33     }
34 }
35
36 $manager = new Manager("Martin", "Sophie", "RH");
37 echo $manager->sePresenter();
38 // Je suis Sophie Martin. Je manage le departement RH.
```

Listing 12 – Héritage et surcharge de méthodes

4 Gestion des entrées utilisateur et sécurité

4.1 Récupérer les données d'un formulaire

```
1 <!-- formulaire.html -->
2 <form method="POST" action="traitement.php">
3     <input type="text" name="nom" placeholder="Votre nom">
4     <input type="email" name="email" placeholder="Votre email">
5     <button type="submit">Envoyer</button>
6 </form>
```

Listing 13 – Formulaire HTML + traitement PHP

```
1 <?php
2 // traitement.php
3 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
4     $nom = $_POST['nom'] ?? '';
5     $email = $_POST['email'] ?? '';
6
7     echo "Nom : $nom | Email : $email";
8 }
```

Listing 14 – Traitement des données POST

4.2 Échappement et sécurité

Attention

Ne jamais faire confiance aux données venant de l'utilisateur ! Une donnée non échappée peut provoquer des failles XSS ou des injections SQL.

```
1 <?php
2 // DANGEREUX : faille XSS
3 echo $_POST['nom'];
4
5 // SECURISE : htmlspecialchars echappe le HTML
6 $nom = htmlspecialchars($_POST['nom'] ?? '', ENT_QUOTES, 'UTF-8');
7 echo $nom;
8
9 // DANGEREUX : injection SQL
10 $id = $_GET['id'];
11 $pdo->query("SELECT * FROM Employe WHERE Id_Employe = $id");
12
13 // SECURISE : requete preparee
14 $id = $_GET['id'] ?? 0;
15 $stmt = $pdo->prepare("SELECT * FROM Employe WHERE Id_Employe = :id");
16 $stmt->execute([':id' => $id]);
17 $employe = $stmt->fetch(PDO::FETCH_ASSOC);
```

Listing 15 – Échappement et requêtes sécurisées

4.3 Validation des données

```
1 <?php
2 $erreurs = [];
3 $nom      = trim($_POST['nom']  ?? '');
4 $email    = trim($_POST['email'] ?? '');
5 $age      = trim($_POST['age']  ?? '');
6
7 if (empty($nom)) {
8     $erreurs[] = "Le nom est obligatoire.";
9 }
10 if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
11     $erreurs[] = "L'email n'est pas valide.";
12 }
13 if (!filter_var($age, FILTER_VALIDATE_INT) || $age < 18) {
14     $erreurs[] = "L'age doit etre un entier superieur a 18.";
15 }
16
17 if (!empty($erreurs)) {
18     foreach ($erreurs as $erreur) {
19         echo "<p style='color:red;'>$erreur</p>";
20     }
21 } else {
22     echo "Formulaire valide !";
23 }
```

Listing 16 – Validation complète d'un formulaire

5 Débogage avec var_dump

5.1 Les outils de débogage

```
1 <?php
2 $employe = [
3     "nom"    => "Dupont",
4     "age"    => 30,
5     "actif"  => true
6 ];
7
8 // var_dump : affiche le TYPE et la VALEUR (le plus complet)
9 var_dump($employe);
10 /*
11 array(3) {
12     ["nom"]    => string(6) "Dupont"
13     ["age"]    => int(30)
14     ["actif"]  => bool(true)
15 }
16 */
17
18 // print_r : plus lisible, sans les types
19 print_r($employe);
20
21 // var_export : representation PHP valide
22 var_export($employe);
```

Listing 17 – var_dump, print_r et var_export

5.2 Fonctions de débogage personnalisées

```
1 <?php
2 // Afficher proprement dans le navigateur
3 function debug($variable) {
4     echo "<pre>";
5     var_dump($variable);
6     echo "</pre>";
7 }
8
9 // Dump and Die : affiche et arrete le script
10 function dd($variable) {
11     echo "<pre>";
12     var_dump($variable);
13     echo "</pre>";
14     die();
15 }
16
17 // Utilisation
18 $data = ["id" => 1, "nom" => "Test"];
19 dd($data); // Affiche et stoppe immediatement
```

Listing 18 – Fonctions debug() et dd()

5.3 Activer l’affichage des erreurs

```
1 <?php
2 // A mettre tout en haut de ton fichier (DEVELOPPEMENT
   UNIQUEMENT)
3 error_reporting(E_ALL);
4 ini_set('display_errors', 1);
```

Listing 19 – Affichage des erreurs en développement

Attention

Ne jamais laisser `display_errors = 1` en production ! Les erreurs pourraient révéler des informations sensibles sur ton serveur.

6 Superglobales

Info

Les superglobales sont des variables spéciales disponibles **partout** dans le code, sans avoir à les passer en paramètre.

Superglobale	Rôle
\$_GET	Données passées dans l'URL
\$_POST	Données envoyées par formulaire
\$_SESSION	Données de session (persistance entre pages)
\$_COOKIE	Données stockées dans le navigateur
\$_FILES	Fichiers téléversés
\$_SERVER	Informations sur le serveur et la requête
\$_ENV	Variables d'environnement
\$GLOBALS	Toutes les variables globales

```
1 <?php
2 // $_GET : URL -> page.php?id=5&nom=Jean
3 $id = $_GET['id'] ?? null;    // 5
4 $nom = $_GET['nom'] ?? null;  // Jean
5
6 // $_SERVER
7 echo $_SERVER['REQUEST_METHOD']; // GET ou POST
8 echo $_SERVER['HTTP_HOST'];      // localhost
9 echo $_SERVER['REQUEST_URI'];    // /mon_projet/page.php?id=5
10 echo $_SERVER['REMOTE_ADDR'];   // IP du visiteur
11
12 // $_SESSION
13 session_start(); // OBLIGATOIRE en tout premier !
14 $_SESSION['utilisateur'] = "Jean";
15 $_SESSION['role']       = "admin";
16
17 // Sur une autre page :
18 session_start();
19 echo $_SESSION['utilisateur']; // Jean
20
21 // Detruire la session (deconnexion)
22 session_destroy();
```

Listing 20 – Exemples de superglobales

7 Téléversement de fichiers

7.1 Formulaire HTML

```
1 <!-- IMPORTANT : enctype="multipart/form-data" est obligatoire
   -->
2 <form method="POST" action="upload.php" enctype="multipart/form-
   data">
3     <input type="file" name="mon_fichier" accept=".pdf,.jpg,.png
       ">
4     <button type="submit">Envoyer</button>
5 </form>
```

Listing 21 – Formulaire avec upload de fichier

7.2 Traitement sécurisé du fichier

```
1 <?php
2 // upload.php
3 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
4
5     if (!isset($_FILES['mon_fichier'])
6         || $_FILES['mon_fichier']['error'] !== UPLOAD_ERR_OK) {
7         die("Erreur lors du telechargement.");
8     }
9
10    $fichier = $_FILES['mon_fichier'];
11    $nom_orig = $fichier['name'];
12    $taille = $fichier['size'];
13    $tmp = $fichier['tmp_name'];
14
15    // 1. Verifier l'extension autorisee
16    $exts_ok = ['pdf', 'jpg', 'jpeg', 'png'];
17    $extension = strtolower(pathinfo($nom_orig,
18    PATHINFO_EXTENSION));
19
20    if (!in_array($extension, $exts_ok)) {
21        die("Type de fichier non autorise.");
22    }
23
24    // 2. Verifier la taille (max 2 Mo)
25    if ($taille > 2 * 1024 * 1024) {
26        die("Fichier trop volumineux (max 2 Mo).");
27    }
28
29    // 3. Generer un nom unique
30    $nouveau_nom = uniqid('fichier_', true) . '.' . $extension;
31
32    // 4. Definir le dossier de destination
33    $destination = __DIR__ . '/uploads/';
34    if (!is_dir($destination)) {
```

```

34     mkdir($destination, 0755, true);
35 }
36
37 // 5. Déplacer le fichier
38 if (move_uploaded_file($tmp, $destination . $nouveau_nom)) {
39     echo "Fichier uploadé : $nouveau_nom";
40
41     // Sauvegarder le chemin en BDD
42     // require_once 'connexion.php';
43     // $stmt = $pdo->prepare(
44     //     "UPDATE Employe SET CV_Path_Employe = :chemin
45     //     WHERE Id_Employe = :id"
46     // );
47     // $stmt->execute([':chemin' => $nouveau_nom, ':id' =>
48     //     1]);
49 } else {
50     echo "Erreur lors du déplacement du fichier.";
51 }

```

Listing 22 – Traitement complet et sécurisé d'un upload

7.3 Téléverser plusieurs fichiers

```

1 <!-- HTML : ajouter l'attribut multiple -->
2 <input type="file" name="fichiers[]" multiple accept=".jpg,.png"
  ">

```

Listing 23 – Upload multiple

```

1 <?php
2 foreach ($_FILES['fichiers']['tmp_name'] as $index => $tmp) {
3     $nom      = $_FILES['fichiers']['name'][$index];
4     $erreur   = $_FILES['fichiers']['error'][$index];
5
6     if ($erreur === UPLOAD_ERR_OK) {
7         $ext      = strtolower(pathinfo($nom,
8             PATHINFO_EXTENSION));
9         $nouveau_nom = uniqid() . '.' . $ext;
10        move_uploaded_file($tmp, __DIR__ . '/uploads/' .
11            $nouveau_nom);
12        echo "Fichier $nouveau_nom uploadé.<br>";
13    }
14 }

```

Listing 24 – Traitement de plusieurs fichiers

8 Connexion à la base de données avec LAMP

Info

Cette section couvre **deux approches complémentaires** : le fichier PHP `connexion.php` pour que ton application se connecte à la base, et l'extension **SQL-Tools** dans VS Code pour explorer et requêter ta base directement depuis l'éditeur.

8.1 Prérequis : vérifier que LAMP est actif

```
1  # Verifier Apache
2  sudo systemctl status apache2
3
4  # Verifier MySQL
5  sudo systemctl status mysql
6
7  # Les demarrer si necessaire
8  sudo systemctl start apache2
9  sudo systemctl start mysql
```

Listing 25 – Vérification et démarrage de LAMP

8.2 Approche 1 — Fichier de connexion PHP

Le fichier `connexion.php` est le **pont entre ton application web et la base de données**. Il doit être créé **une seule fois** et inclus dans tous les fichiers qui ont besoin de la base.

8.2.1 Créer le fichier `connexion.php`

Place ce fichier à la racine de ton projet : `/var/www/html/ton_projet/connexion.php`

```
1  <?php
2  $host    = '127.0.0.1';           // ou 'localhost'
3  $dbname  = 'nom_de_ta_bdd';      // nom exact de ta base dans
    phpMyAdmin
4  $user    = 'ton_utilisateur';    // utilisateur MySQL
5  $password = 'ton_mot_de_passe';
6
7  try {
8      $pdo = new PDO(
9          "mysql:host=$host;dbname=$dbname;charset=utf8",
10         $user,
11         $password
12     );
13     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
14     // Connexion reussie (pas de message en production)
15 } catch (PDOException $e) {
16     die("Erreur de connexion : " . $e->getMessage());
```

```
17 }  
18 ?>
```

Listing 26 – connexion.php — fichier de connexion PDO

8.2.2 Tester la connexion dans le navigateur

Crée un fichier `test_connexion.php` :

```
1 <?php  
2 require_once 'connexion.php';  
3 echo "Connexion reussie a la base : " . $dbname;  
4 ?>
```

Listing 27 – test_connexion.php

Puis ouvre dans ton navigateur :

`http://localhost/ton_projet/test_connexion.php`

8.2.3 Utiliser la connexion dans les autres fichiers

```
1 <?php  
2 require_once 'connexion.php'; // Inclure la connexion  
3  
4 // SELECT      Recuperer tous les employes  
5 $stmt = $pdo->query("SELECT * FROM Employee");  
6 $employees = $stmt->fetchAll(PDO::FETCH_ASSOC);  
7  
8 foreach ($employees as $e) {  
9     echo $e['Nom_Employe'] . " " . $e['Prenom_Employe'] . "<br>"  
10    ;  
11 }  
12  
13 // INSERT      Ajouter un employe  
14 $stmt = $pdo->prepare(  
15     "INSERT INTO Employee (Nom_Employe, Prenom_Employe)  
16     VALUES (:nom, :prenom)"  
17 );  
18 $stmt->execute([':nom' => 'Dupont', ':prenom' => 'Jean']);  
19  
20 // UPDATE      Modifier un employe  
21 $stmt = $pdo->prepare(  
22     "UPDATE Employee SET Nom_Employe = :nom WHERE Id_Employe = :  
23     id"  
24 );  
25 $stmt->execute([':nom' => 'Martin', ':id' => 1]);  
26  
27 // DELETE      Supprimer un employe  
28 $stmt = $pdo->prepare("DELETE FROM Employee WHERE Id_Employe = :  
29     id");  
30 $stmt->execute([':id' => 1]);
```

28 ?>

Listing 28 – Utilisation de connexion.php dans un fichier

8.2.4 Afficher une table dans le navigateur

```

1  <?php
2  require_once 'connexion.php';
3  $stmt      = $pdo->query("SELECT * FROM Employe");
4  $employees = $stmt->fetchAll(PDO::FETCH_ASSOC);
5  ?>
6  <!DOCTYPE html>
7  <html lang="fr">
8  <head>
9      <meta charset="UTF-8">
10     <title>Liste des Employes</title>
11 </head>
12 <body>
13 <h1>Liste des Employes</h1>
14 <table border="1">
15     <thead>
16         <tr>
17             <th>ID</th>
18             <th>Nom</th>
19             <th>Prenom</th>
20             <th>Contrat</th>
21         </tr>
22     </thead>
23     <tbody>
24         <?php foreach ($employees as $e) : ?>
25         <tr>
26             <td><?= $e['Id_Employe'] ?></td>
27             <td><?= htmlspecialchars($e['Nom_Employe']) ?></td>
28             <td><?= htmlspecialchars($e['Prenom_Employe']) ?></td>
29             <td><?= htmlspecialchars($e['Type_De_Contrat_Employe
30                 ']) ?></td>
31         </tr>
32         <?php endforeach; ?>
33     </tbody>
34 </table>
35 </body>
36 </html>

```

Listing 29 – employees.php — Affichage d'une table en HTML

Accès depuis le navigateur :

http://localhost/ton_projet/employees.php

8.3 Approche 2 — Extension SQLTools dans VS Code

SQLTools est un outil **visuel pour le développeur**. Il ne remplace pas le fichier PHP, mais permet d'explorer et tester ta base **sans quitter VS Code**.

8.3.1 Extensions à installer

Dans VS Code (Ctrl+Shift+X), installe les deux extensions suivantes :

Extension	Rôle
SQLTools	Interface principale
SQLTools MySQL/MariaDB/TiDB	Driver pour se connecter à MySQL

8.3.2 Configurer la connexion

1. Clique sur l'icône SQLTools dans la barre latérale (icône cylindre)
2. Clique sur **“Add New Connection”**
3. Choisis **MySQL / MariaDB**
4. Remplis les champs :

Champ	Valeur
Connection Name	MonProjet (au choix)
Server	127.0.0.1
Port	3306
Database	nom_de_ta_bdd
Username	ton_utilisateur
Password	ton_mot_de_passe

5. Clique sur **“Test Connection”**
6. Si le test est réussi → clique sur **“Save Connection”**

8.3.3 Explorer les tables

Une fois connecté, dans le panneau SQLTools :

```
1 Connexion MonProjet
2   nom_de_ta_bdd
3     Tables
4       Employe
5       Entreprise
6       Utilisateur
7       Mission
8       ...
```

Listing 30 – Arborescence SQLTools

Clique sur une table → icône **Show Records** → tu vois toutes les lignes.

8.3.4 Exécuter des requêtes SQL depuis VS Code

Ouvre un nouveau fichier `.sql` dans VS Code et exécute directement :

```
1  -- Voir tous les employes
2  SELECT * FROM Employe;
3
4  -- Voir les missions actives
5  SELECT * FROM Mission WHERE Statut_Mission = 'actif';
6
7  -- Compter les utilisateurs
8  SELECT COUNT(*) AS total FROM Utilisateur;
```

Listing 31 – Requêtes SQL dans VS Code via SQLTools

Pour exécuter : **Ctrl+Enter** sur la ligne souhaitée.

8.3.5 Résumé comparatif

	connexion.php	SQLTools
Utilisé par	Ton application web	Toi, le développeur
Obligatoire	Oui	Non (confort)
Accès	Via le navigateur	Via VS Code
Utilité	Faire fonctionner le site	Explorer / tester la BDD

Bonne pratique

Utilise **SQLTools** pour tester tes requêtes SQL rapidement, puis copie-les dans tes fichiers PHP avec `require_once 'connexion.php'` pour les intégrer à ton application.

Récapitulatif général

Module	Ce que tu sais faire
1 — Syntaxe	Variables, conditions, boucles, fonctions
2 — Chaînes	Quotes, concaténation, <code>mb_*</code> , fonctions utiles
3 — POO	Classes, objets, héritage, encapsulation
4 — Sécurité	<code>htmlspecialchars</code> , requêtes préparées, validation
5 — Débogage	<code>var_dump</code> , <code>print_r</code> , <code>dd()</code> , affichage des erreurs
6 — Superglobales	<code>\$_GET</code> , <code>\$_POST</code> , <code>\$_SESSION</code> , <code>\$_SERVER</code> , <code>\$_FILES</code>
7 — Téléversement	Upload simple et multiple, sécurisation
8 — Connexion BDD	<code>connexion.php</code> (PDO), SQLTools (VS Code)